

4 Project IV: Diffusion Models

Diffusion probabilistic models have recently become popular in Computer Vision due to their ability to generate high-resolution realistic images. In this project, you will be diving deep into the actual working of such models. You will work your way through the math involved and the tricks used to train neural network models to accomplish the generative task using this method. Due to limited data and computation available, you will be working with simple toy data.

Notes

1. It is recommended to use [PyTorch](#) for mandatory part-2 and the advanced part.
2. [blog 1](#) and [blog 2](#) are excellent sources that can serve as tutorials for the details of how diffusion models work and how they can be implemented.

4.1 Introduction

As discussed in the lectures, a (denoising) diffusion model consists of 2 parts:

1. A pre-defined forward diffusion process q that gradually adds small amounts of Gaussian noise to the original data, until you end up with pure noise.
2. A learned reverse (denoising) model p_θ that removes the noise to eventually produce something that if the training is successful should look like a sample from the training distribution.

Assuming a data point to have been sampled from an original distribution $\mathbf{x}_0 \sim q(\mathbf{x})$. We sequentially add Gaussian noise to it at timesteps $1 \dots T$ as per the conditional distribution:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad (1)$$

where the step sizes are controlled by the variance schedule $\{\beta_t \in (0, 1)\}_{t=1}^T$.

4.1.1 Mandatory task 1

If somehow the reverse process $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ was available, one would be able to produce meaningful data samples by repeatedly applying this denoising process starting from pure noise. Here we will consider a simple 1D case: $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{x}_0; m, p)$ and the forward diffusion process as mentioned above in (1). Assuming that the distribution of $\mathbf{x}_0$ is known, the reverse conditional distribution $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ can be computed in closed form, using the Bayes rule and the product of Gaussian distributions.

- Choose your own parameters (m, p) for the distribution of $\mathbf{x}_0$ and sample from this. Choose m and p such that you do not get the standard normal distribution (since we want a distribution that is different to the distribution used for x_T).
- Run the forward diffusion process for a number of timesteps, $T = 50$. You can experiment with different variance schedules (eg. linear, quadratic, etc.), but for simplicity, you can start with a constant schedule such that $\beta_t = 0.25 \forall t \in \{1 \dots T\}$. By repeating this process for various inputs, verify that the values at time T approximately follow a standard normal distribution. Visualize a few forward trajectories and also the distribution of x_T computed over at least 100 trajectories starting from different realizations of $\mathbf{x}_0$ by plotting x_t against t for $t \in (1 \dots T)$.

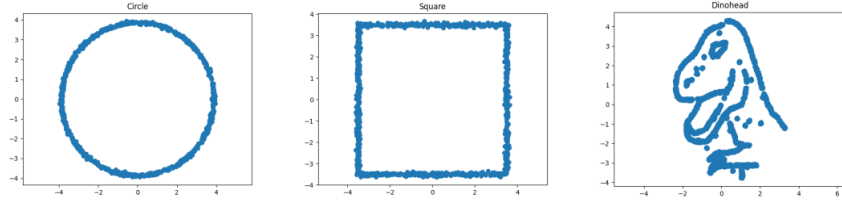


Figure 6: 2D point cloud datasets for mandatory task 2 - *Circle*, *Square* and *Dinohead*

- Compute the reverse process as a function of $\mathbf{x}_t$, $\mathbf{x}_0$ and t . You can take help from the derivations mentioned [here](#).
- Sample $\mathbf{x}_T$ from a standard normal distribution and repeatedly apply the reverse process computed above to obtain $\mathbf{x}_0$. Visualize a few of the reverse trajectories.

4.1.2 Mandatory task 2

In this task you will actually predict the $q(\mathbf{x}_{t-1}|\mathbf{x}_t)$ using a neural network with parameters θ . Hence this prediction/approximation will be termed as $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$. To learn this, you will closely follow the approach presented in [1]. You are provided sets of 2D points in the following shapes - a *Circle*, a *Square*, and *Dinohead* - an outline of a (rather simplistic) dinosaur head (Fig. 6). The task is to learn to produce something close to these starting from standard Gaussian noise. You are supposed to train a different model for each shape. This is not a great example to showcase the generative ability of diffusion methods, however, it is interesting in the sense that it allows understanding and implementing the intrinsic details of the method.

Use the following steps as guidelines to implement a training regime that produces the required output. While training the model for a particular shape example, do the following within each training loop:

- Sample a batch (randomly selected small subset) of points and for each point - a timestep, and noise - this is the true noise.
- Predict the 2D noise $\epsilon_\theta(\mathbf{x}_t, t)$ using $\mathbf{x}_t$ and t as inputs.
- Using squared error between the predicted and true noise as the loss, back-propagate and update the model parameters using gradient descent.

Other general tips:

- Instead of feeding the 2D points and the timesteps as inputs to the neural network directly, it is advisable to feed in *positional embeddings* of these inputs. High-frequency sinusoidal embeddings perform well. A python class extending `torch.nn.Module` for sinusoidal embeddings is provided in the *Stuff_for_Diffusion_Models_Project* folder on Canvas. You may use it as reference as it might or might not match your required format exactly.
- An MLP (multi-layer perceptron) of 3-4 layers should be able to accomplish this task, trained for enough steps. Training such an MLP with Adam optimizer and learning rate in the range of 10^{-3} to 10^{-4} has been found to be effective.

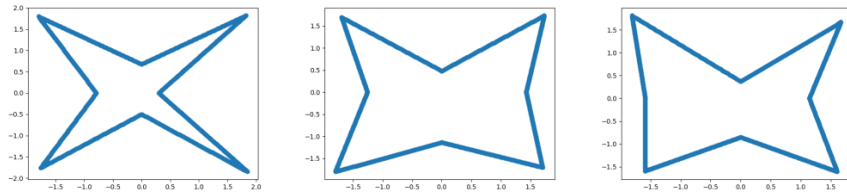


Figure 7: Examples of data samples from the *stars* dataset.

What to include in the report

- An introduction section describing the basics of diffusion probabilistic models, the underlying distributions, and the applications of such methods. This section should also explain the details of how the denoising task can be accomplished by a noise-predicting neural network. (About 1-2 pages)
- A method section with 2 subsections for the 2 mandatory tasks.
 1. In the first subsection, explain in your own words how the reverse process can be computed in closed form in the given setting.
 2. In the second subsection, explain in detail the steps involved in training your neural network and list the hyper-parameters used. Also, explain the steps involved in generating data at inference time.
- An experimental results section, again containing two subsections.
 1. For the first task, include the plots for forward, and backward trajectories and also the resulting distributions (of $\mathbf{x}_T$ in the forward process and $\mathbf{x}_0$ in the reverse process).
 2. For the second task, include scatter plots of the resulting 2D points from your inference process (reverse sampling from 2D Gaussian noise). Show results obtained at different stages of the training process (eg. after every 1k training steps). Also, include analysis over any other hyper-parameter whose impact you found interesting.

4.2 Advanced part

In the advanced part, you attempt to generate (somewhat) new data. Like 4.1.2, the data is still in the form of 2D points, however, instead of learning to generate points in a single pattern, you will try to generate different realizations of a shape template (examples shown in figure 7). You are provided 1024 data samples with 1024 points each. You can devise your own strategy for this or you can have roughly the same approach as before except the following major change.

1. Instead of your network producing noise for each point independently, the network would take in information about all points in the batch to estimate the noise for the batch.
Tip : Since the output noise should permute similarly upon the input points being permuted in a certain way, the additional information from other points should be something that is permutation invariant - like the average position of the points.

Upon successful implementation, you should be able to generate samples which resemble the input data (perhaps noisy versions of them).

4.3 Theoretical Part

Answer the following questions in your report. Use your own words and justify your answers.

(a) Compute the theoretical number of learnable parameters in your model. Show explicitly how the parameter count is derived for each relevant layer or component. Compare your theoretical result with the parameter count reported by the implementation, framework, or documentation, and discuss possible reasons for any differences. Discuss how the number of learnable parameters would change if you modified the architecture, for example by changing the hidden dimensionality or the number of hidden layers. Finally, discuss whether increasing the number of learnable parameters would necessarily improve the generated samples in this project.

(b) Estimate the computational complexity of your pipeline for (i) training, and (ii) inference/sampling. Clearly state all assumptions, including implementation details. Discuss which parts of the pipeline dominate the computational cost. Discuss why inference in diffusion models can be computationally expensive even when the underlying denoising model is relatively small.

(c) Identify 1-2 qualitative artifacts in your generated results in mandatory task 2 that deviates noticeably from the geometric structure or visual appearance of the corresponding training shape. Explain how the occurrence of these artifacts may be related to the assumption that points are generated independently. Discuss what types of geometric or structural relationships between points are difficult to capture under the independent-point assumption. Explain how such artifacts could potentially be reduced or avoided by training models on entire point clouds jointly, for example as in the advanced part of the project.

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arxiv:2006.11239*, 2020. [13](#)

4.3 Theoretical Part

Answer the following questions in your report. Use your own words and justify your answers.

(a) Compute the theoretical number of learnable parameters in your model. Show explicitly how the parameter count is derived for each relevant layer or component. Compare your theoretical result with the parameter count reported by the implementation, framework, or documentation, and discuss possible reasons for any differences. Discuss how the number of learnable parameters would change if you modified the architecture, for example by changing the hidden dimensionality or the number of hidden layers. Finally, discuss whether increasing the number of learnable parameters would necessarily improve the generated samples in this project.

(b) Estimate the computational complexity of your pipeline for (i) training, and (ii) inference/sampling. Clearly state all assumptions, including implementation details. Discuss which parts of the pipeline dominate the computational cost. Discuss why inference in diffusion models can be computationally expensive even when the underlying denoising model is relatively small.

(c) Identify 1-2 qualitative artifacts in your generated results in mandatory task 2 that deviates noticeably from the geometric structure or visual appearance of the corresponding training shape. Explain how the occurrence of these artifacts may be related to the assumption that points are generated independently. Discuss what types of geometric or structural relationships between points are difficult to capture under the independent-point assumption. Explain how such artifacts could potentially be reduced or avoided by training models on entire point clouds jointly, for example as in the advanced part of the project.

References

- [1] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *arXiv preprint arxiv:2006.11239*, 2020. [13](#)